

Technical Summary: Automatic Chain-of-Thought Prompting in Large Language Models

Kai-Yu Lu

1 Research Problem and Motivation

This study investigates how to elicit reliable multi-step reasoning from large language models (LLMs) by *automatic* construction of chain-of-thought (CoT) demonstrations for in-context learning (ICL). Chain-of-thought prompting encourages an LLM to generate intermediate reasoning steps in natural language before producing a final answer.

Existing CoT prompting paradigms can be grouped into two major categories.

- **Zero-Shot-CoT** appends a simple instruction such as “Let us think step by step” after a test question so that the LLM produces a step-by-step reasoning chain and then an answer.
- **Manual-CoT** uses a few handcrafted CoT demonstrations. Each demonstration consists of a question, a human-written reasoning chain, and a correct answer. These demonstrations are concatenated one-by-one before the test question.

Manual-CoT typically achieves stronger performance than Zero-Shot-CoT on arithmetic, commonsense, and symbolic reasoning benchmarks. However, its success depends on careful human design of task-specific demonstration questions and reasoning chains. This manual design is expensive and must be repeated for each new task.

At the same time, Zero-Shot-CoT alone is insufficient. Although Zero-Shot-CoT reveals that LLMs are decent zero-shot reasoners, the generated reasoning chains and answers can contain systematic errors. A naive idea is to automatically build demonstrations by retrieving questions similar to a test question and then generating reasoning chains for these retrieved questions via Zero-Shot-CoT. The paper shows that this intuitive strategy fails due to a phenomenon called *misleading by similarity*, where semantically similar questions tend to share similar failure patterns.

The central research problem is therefore:

How to automatically construct CoT demonstrations from LLM outputs themselves, using only unlabeled questions, such that the resulting prompts match or exceed the performance of manually designed CoT demonstrations.

The proposed solution is **Auto-CoT** (Automatic Chain-of-Thought prompting), which samples questions with diversity and uses Zero-Shot-CoT to generate reasoning chains for these questions, thereby constructing demonstrations without any manual rationales.

2 Related Work

2.1 Chain-of-thought prompting

Chain-of-thought prompting is a technique that elicits explicit reasoning chains from LLMs. The original CoT work showed that few-shot prompts with carefully written question–rationale–answer examples substantially improve multi-step reasoning performance on math word problems and other tasks.

Zero-Shot-CoT demonstrated that a single generic cue such as “Let us think step by step” appended to the test question can already trigger useful reasoning behavior without any demonstrations. Manual-CoT further improves performance by providing informative demonstrations one-by-one with detailed reasoning.

Subsequent lines of work extend CoT prompting in two main directions.

- **Problem decomposition.** Least-to-most prompting decomposes a complex problem into simpler sub-problems and solves them sequentially. Each step uses CoT reasoning for a smaller sub-task, and the partial solutions are combined.
- **Voting over reasoning paths.** Self-consistency decoding samples multiple CoT reasoning paths for the same question and takes a majority vote over the final answers. Other methods introduce randomness at the input level to generate diverse rationales for ensemble-style decision making.

These approaches rely on manually-designed demonstrations as seeds and mainly aim to enhance reasoning once good demonstrations are already available. In contrast, Auto-CoT targets the elimination of manual design itself while maintaining competitive performance.

2.2 In-context learning and demonstration selection

In-context learning (ICL) enables LLMs to perform a target task by conditioning on a small number of labeled examples in the prompt, without parameter updates. Many studies show that ICL performance is highly sensitive to the choice and formatting of demonstrations, including the selection of examples, their order, and their lexical realization.

Similarity-based retrieval methods select demonstrations whose inputs are close to the test question in an embedding space, for example using Sentence-BERT encoders and cosine similarity. These methods have been successful in classification tasks with annotated labels.

However, most prior analyses focus on direct input-to-label mappings. CoT prompting introduces a richer structure of input-to-rationale-to-answer. This work shows that in the presence of noisy, LLM-generated rationales, similarity-based retrieval can be harmful because similar questions are more likely to share correlated reasoning errors.

3 Dataset Construction

The method is evaluated on ten public benchmark datasets that cover three categories of reasoning tasks: arithmetic reasoning, commonsense reasoning, and symbolic reasoning. No new dataset is introduced. Table 1 summarizes the dataset statistics as reported in the paper.

3.1 Arithmetic reasoning

Arithmetic reasoning is evaluated on six datasets:

- **MultiArith** consists of math word problems that require multiple arithmetic operations such as addition, subtraction, multiplication, and division to reach a numeric answer.

Dataset	Number of samples	Average words	Answer format	Licence
MultiArith	600	31.8	Number	Unspecified
AddSub	395	31.5	Number	Unspecified
GSM8K	1319	46.9	Number	MIT License
AQUA	254	51.9	Multiple choice	Apache-2.0
SingleEq	508	27.4	Number	No License
SVAMP	1000	31.8	Number	MIT License
CSQA	1221	27.8	Multiple choice	Unspecified
StrategyQA	2290	9.6	Yes or No	Apache-2.0
Last Letters	500	15.0	String	Unspecified
Coin Flip	500	37.0	Yes or No	Unspecified

Table 1: Dataset description for arithmetic, commonsense, and symbolic reasoning tasks.

- **AddSub** contains word problems that primarily involve addition and subtraction of quantities in everyday contexts.
- **GSM8K** (Grade School Math 8K) includes grade-school-level math problems that typically require several steps of reasoning and intermediate calculations to produce a number.
- **AQUA-RAT** (Algebra Question Answering with Rationales) is a multiple-choice dataset where questions often require algebraic reasoning; the evaluation in the paper uses the AQUA subset.
- **SingleEq** is composed of word problems solvable by forming a single linear equation.
- **SVAMP** contains adversarially constructed math word problems designed to test robustness beyond shallow pattern matching.

All these tasks expect a single numeric answer or a choice among options.

3.2 Commonsense reasoning

Commonsense reasoning is evaluated on:

- **CommonsenseQA (CSQA)**, which provides multiple-choice questions requiring background knowledge and non-trivial semantic understanding.
- **StrategyQA**, which contains yes/no questions requiring implicit multi-hop reasoning over several facts, often involving strategic considerations.

Answers are discrete options or binary decisions.

3.3 Symbolic reasoning

Symbolic reasoning benchmarks focus on deterministic rule-based transformations.

- **Last Letter Concatenation** (Last Letters) requires taking a sequence of words and outputting a string formed by concatenating the last letter of each word in order.
- **Coin Flip** simulates a sequence of people who may flip a coin or leave it as is. Starting from heads, the task is to predict whether the coin shows heads or tails at the end, often formatted as yes/no.

These tasks stress symbolic manipulation rather than world knowledge.

4 Query Protocol and Task Definitions

4.1 Prompting protocols

The paper compares several prompting protocols that differ in how demonstrations and CoT cues are used.

- **Zero-Shot** concatenates a test question with a short template such as “The answer is” and asks the LLM to directly output the answer without explicit reasoning.
- **Zero-Shot-CoT** appends the CoT cue “Let us think step by step” to the test question. The LLM is expected to produce a multi-step rationale followed by a final answer.
- **Few-Shot** provides several manually written question–answer pairs *without* rationales. These demonstrations are concatenated before the test question.
- **Manual-CoT** uses manually designed CoT demonstrations. Each demonstration has the form “Q: question. A: step-by-step rationale. Therefore, the answer is ...” and is concatenated before the test question together with the CoT cue.
- **Random-Q-CoT** automatically constructs demonstrations by sampling questions uniformly at random from the dataset (excluding the test question). For each sampled question, Zero-Shot-CoT generates a rationale and answer, which are used as a demonstration.
- **Retrieval-Q-CoT** automatically constructs demonstrations by retrieving questions similar to the test question using a Sentence-BERT encoder and cosine similarity. Zero-Shot-CoT generates rationales and answers for the retrieved questions.
- **Auto-CoT** is the proposed method. It clusters questions, selects one representative question from each cluster, generates a reasoning chain via Zero-Shot-CoT, and applies heuristics to filter long or complex demonstrations.

For all CoT-based protocols, the LLM outputs free-form text containing both a rationale and an answer. A deterministic post-processing rule extracts the predicted answer from the last numeric expression, the selected option, or the final yes/no phrase.

4.2 Task definitions and evaluation

For each dataset, the task can be framed as mapping a natural language question q to a textual answer a .

- Arithmetic and symbolic datasets require a single numeric or symbolic answer (for example, a number or a concatenated string).
- Multiple-choice datasets require selecting the correct option among a fixed set of alternatives.
- Yes/no datasets require a binary decision.

The main evaluation metric is **accuracy**, defined as the percentage of test questions whose extracted prediction exactly matches the ground-truth answer. No other metrics such as F1-score are used in the paper.

For the analysis of automatic demonstration construction, the study also defines:

- **Error rate** of Zero-Shot-CoT within a subset of questions, given by the proportion of questions whose Zero-Shot-CoT predictions are incorrect.

- **Unresolving rate** of a demonstration construction method, computed on the subset of questions where Zero-Shot-CoT fails. It is the fraction of these questions that remain incorrectly answered when the method is applied. A higher unresolving rate indicates that the method is more likely to preserve mistakes made by Zero-Shot-CoT.

5 Modeling Approach

5.1 Failure of similarity-based automatic CoT prompting

Before introducing Auto-CoT, the authors study whether simple automatic constructions based on Zero-Shot-CoT can replace manually-written demonstrations.

Random-Q-CoT and Retrieval-Q-CoT are designed as follows.

- **Random-Q-CoT** chooses k demonstration questions uniformly at random from the dataset (excluding the test question). Zero-Shot-CoT is applied to each demonstration question to generate a reasoning chain and answer. These k triples form the CoT demonstrations.
- **Retrieval-Q-CoT** encodes all questions with Sentence-BERT and uses cosine similarity to retrieve the top- k nearest questions to the test question. Zero-Shot-CoT generates reasoning chains and answers for these retrieved questions, which are then used as demonstrations.

On the arithmetic dataset MultiArith, an unexpected result is observed. When demonstrations are constructed from *ground-truth* reasoning chains (available from annotated training sets), Retrieval-Q-CoT outperforms Random-Q-CoT and even Manual-CoT. However, when demonstrations use *Zero-Shot-CoT-generated* reasoning chains, Retrieval-Q-CoT underperforms Random-Q-CoT.

The authors analyze this behavior with the notion of *unresolved questions*. Starting from a subset Q of MultiArith test questions that Zero-Shot-CoT answers incorrectly, they measure how many of these questions remain incorrect when Random-Q-CoT or Retrieval-Q-CoT is used. The results show:

- Retrieval-Q-CoT has an unresolving rate of 46.9%.
- Random-Q-CoT has a lower unresolving rate of 25.8%.

This indicates that similarity-based retrieval, when combined with noisy Zero-Shot-CoT rationales, tends to *propagate* reasoning mistakes: similar questions are likely to share similar error patterns, so using them as demonstrations causes the model to repeat the same errors. This phenomenon is termed **misleading by similarity**.

5.2 Question clustering and frequent-error clusters

To further understand where Zero-Shot-CoT fails, the authors cluster questions with Sentence-BERT embeddings and k -means clustering.

- For each dataset, questions are partitioned into k clusters (for example, $k = 8$ on MultiArith).
- For each cluster, the error rate of Zero-Shot-CoT is computed.

The analysis shows that errors are not uniformly distributed. Instead, certain clusters exhibit much higher Zero-Shot-CoT error rates, forming *frequent-error clusters*. For example, in MultiArith, some clusters have relatively low error rates, while one cluster has a significantly higher rate.

If demonstrations come predominantly from a frequent-error cluster, then many demonstrations will contain similar mistakes. Retrieval-Q-CoT, which tends to sample questions close to the test question, is more likely to be restricted to such problematic clusters when a test question itself falls into a frequent-error region.

These observations motivate a design principle: **diversity across clusters**. Instead of focusing demonstrations near the test question, Auto-CoT aims to sample one representative question from each cluster, thereby covering a range of reasoning patterns and reducing the influence of clustered errors.

5.3 Question clustering stage

Given a dataset with question set Q and a desired number of demonstrations k , Auto-CoT first performs question clustering.

- Each question $q \in Q$ is encoded as a vector using Sentence-BERT.
- A k -means algorithm partitions these vectors into k clusters.

For cluster $i \in \{1, \dots, k\}$, the questions assigned to this cluster are sorted in ascending order of their distance to the cluster center, yielding an ordered list

$$q^{(i)} = [q_1^{(i)}, q_2^{(i)}, \dots]. \quad (1)$$

In Equation (1):

- $q^{(i)}$ denotes the ordered list of questions in cluster i .
- $q_1^{(i)}$ is the question closest to the center of cluster i , and $q_2^{(i)}$ is the second closest, and so on.
- The ordering reflects how central a question is within its cluster. More central questions are considered first as candidates for demonstrations.

This stage does not involve any labels or rationales. It purely organizes questions by semantic similarity to prepare for diversity-based sampling.

5.4 Demonstration sampling stage

The second stage of Auto-CoT constructs one demonstration from each cluster, using Zero-Shot-CoT to generate reasoning chains and simple heuristics to filter undesirable candidates.

For each cluster i :

- The algorithm iterates over the ordered questions $q_1^{(i)}, q_2^{(i)}, \dots$.
- For a candidate question $q_j^{(i)}$, a prompt of the form

“Q: $q_j^{(i)}$. A: Let us think step by step.”

is fed to the LLM using Zero-Shot-CoT.

- The LLM outputs a reasoning chain, denoted $r_j^{(i)}$, followed by a final answer, denoted $a_j^{(i)}$.

A candidate demonstration is then constructed as

$$d_j^{(i)} = [\text{Q: } q_j^{(i)}, \text{ A: } r_j^{(i)} \circ a_j^{(i)}], \quad (2)$$

where:

- $d_j^{(i)}$ is the candidate demonstration for cluster i based on question $q_j^{(i)}$.
- $r_j^{(i)}$ is the natural-language rationale produced by Zero-Shot-CoT.
- $a_j^{(i)}$ is the extracted final answer.
- The symbol $\circ$ denotes textual concatenation of the rationale and the answer into the answer field of the demonstration.

To ensure that demonstrations are concise and readable, Auto-CoT imposes simple selection criteria:

- The question $q_j^{(i)}$ must contain no more than 60 tokens.
- The rationale $r_j^{(i)}$ must contain no more than 5 reasoning steps, where a step is approximated by a line separated by a newline character.

If $d_j^{(i)}$ satisfies these criteria, it is accepted as the representative demonstration $d^{(i)}$ for cluster i and the search in that cluster stops. If not, the algorithm considers the next question $q_{j+1}^{(i)}$ in the ordered list.

After all clusters have contributed one accepted demonstration, the final demonstration list is:

$$[d^{(1)}, d^{(2)}, \dots, d^{(k)}].$$

5.5 Using Auto-CoT for in-context reasoning

Given a test question q_{test} , the full LLM input for Auto-CoT is constructed by concatenating all demonstrations, followed by the test question and the CoT cue, with the same pattern used in Manual-CoT. Conceptually, the prompt has the form:

- k demonstration blocks of the form “Q: question. A: step-by-step rationale and final answer.”, one from each cluster.
- A final block “Q: q_{test} . A: Let us think step by step.” that asks the LLM to reason about the test question.

The LLM produces a reasoning chain and final answer for q_{test} , and the answer is extracted by the same post-processing rule as in other CoT protocols.

No loss function is explicitly optimized in this procedure. Instead, Auto-CoT aims to arrange the context so that the LLM’s implicit reasoning ability is better utilized, under the constraints of noisy Zero-Shot-CoT-generated rationales.

5.6 Handling wrong demonstrations and streaming extension

Because Zero-Shot-CoT is imperfect, some automatically constructed demonstrations are wrong, meaning that their answers do not match the ground truth. The paper investigates how such wrong demonstrations affect performance.

- An **In-Cluster Sampling** baseline draws all demonstrations for a test question from the same cluster to which the test question belongs, in order to simulate the effect of low diversity.
- Experiments on MultiArith show that as the proportion of wrong demonstrations increases, In-Cluster Sampling degrades quickly, while Auto-CoT, which samples across clusters, remains more robust.

The authors also consider a more challenging **streaming setting**, where test questions arrive in small batches and the full set of questions is not available in advance. An extension called **Auto-CoT*** incrementally builds a pool of question–rationale–answer triples from earlier batches and periodically clusters them to construct demonstrations for later batches. Results on MultiArith show that:

- For the first batch, Auto-CoT* reduces to Zero-Shot-CoT since no demonstrations exist yet.
- From the second batch onward, Auto-CoT* quickly becomes comparable to Manual-CoT on MultiArith and remains close for subsequent batches.

This indicates that Auto-CoT can be adapted to streaming scenarios while preserving its main benefits.

6 Empirical Results

6.1 Experimental setup

The primary LLM used in experiments is **GPT-3** (text-davinci-002) with 175B parameters. This model has been reported to have strong CoT reasoning performance in prior work.

The study also evaluates the **Codex** model (code-davinci-002) for a subset of datasets to test the generality of Auto-CoT across architectures.

Following Wei et al. (Manual-CoT), the number of demonstrations k is:

- $k = 8$ for most datasets.
- $k = 4$ for AQuA and Last Letters.
- $k = 7$ for CSQA.
- $k = 6$ for StrategyQA.

6.2 Accuracy on ten datasets with GPT-3

Table 2 reports accuracy for Zero-Shot, Zero-Shot-CoT, Few-Shot, Manual-CoT, and Auto-CoT across the ten datasets when using GPT-3 (text-davinci-002).

Key observations from Table 2:

- **Effect of CoT prompting.** Zero-Shot-CoT substantially improves over Zero-Shot on most datasets, showing the benefit of explicit reasoning chains. For example, on Coin Flip, accuracy improves from 53.8% to 91.4%, and on GSM8K from 12.5% to 40.7%.

Model	Arithmetic						Commonsense		Symbolic	
	Multi	GSM8K	AddSub	AQUA	SingleEq	SVAMP	CSQA	Strategy	Letter	Coin
Zero Shot	22.7	12.5	77.0	22.4	78.7	58.8	72.6	54.3	0.2	53.8
Zero Shot CoT	78.7	40.7	74.7	33.5	78.7	63.7	64.6	54.8	57.6	91.4
Few Shot	33.8	15.6	83.3	24.8	82.7	65.7	79.5	65.9	0.2	57.2
Manual CoT	91.7	46.9	81.3	35.8	86.6	68.9	73.5	65.4	59.0	97.2
Auto CoT	92.0	47.9	84.8	36.5	87.0	69.5	74.4	65.4	59.7	99.9

Table 2: Accuracy (%) on ten datasets with GPT 3. Multi denotes MultiArith and Letter denotes Last Letter Concatenation.

- **Effect of manual CoT demonstrations.** Manual-CoT further improves performance over Zero-Shot-CoT, demonstrating the value of well-crafted examples. On MultiArith, accuracy increases from 78.7% (Zero-Shot-CoT) to 91.7% (Manual-CoT).
- **Auto-CoT vs. Manual-CoT.** Auto-CoT consistently matches or slightly exceeds Manual-CoT on all ten datasets. For example:
 - MultiArith: 92.0% (Auto-CoT) vs. 91.7% (Manual-CoT).
 - AddSub: 84.8% (Auto-CoT) vs. 81.3% (Manual-CoT).
 - Coin Flip: 99.9% (Auto-CoT) vs. 97.2% (Manual-CoT).

These results show that fully automatic construction of CoT demonstrations based on clustering and Zero-Shot-CoT rationales can reach or surpass the performance of human-designed CoT prompts across diverse reasoning tasks.

6.3 Accuracy with the Codex LLM

To test generality across LLM backbones, the study also evaluates Auto-CoT and baselines using the Codex model (code-davinci-002) on three arithmetic datasets. The results are summarized in Table 4 of the paper and can be paraphrased as follows:

- On **MultiArith**, Zero-Shot-CoT achieves 64.8% accuracy, Manual-CoT reaches 96.8%, and Auto-CoT achieves 93.2%.
- On **GSM8K**, Zero-Shot-CoT achieves 31.8%, Manual-CoT reaches 59.4%, and Auto-CoT further improves to 62.8%.
- On **AddSub**, Zero-Shot-CoT achieves 65.6%, Manual-CoT reaches 84.6%, and Auto-CoT achieves 91.9%.

These results indicate that Auto-CoT remains competitive or superior even when the underlying LLM changes from GPT-3 to Codex, suggesting that the automatic demonstration construction procedure is model-agnostic.

6.4 Effect of diversity and wrong demonstrations

The paper examines how different demonstration construction strategies behave as the number of wrong demonstrations increases.

On MultiArith, the authors compare Auto-CoT with an In-Cluster Sampling baseline that samples all demonstrations from the cluster containing the test question. The experiments show:

- When only a small fraction of demonstrations are wrong, both methods maintain relatively high accuracy.
- As the proportion of wrong demonstrations increases, In-Cluster Sampling degrades more severely, while Auto-CoT with cluster-level diversity remains more robust.

This supports the central claim that diversity across clusters mitigates the negative effects of noisy Zero-Shot-CoT rationales. Even if one out of eight demonstrations is wrong, the diverse set of demonstrations still covers useful reasoning patterns that guide the model toward correct answers.

6.5 Streaming setting and Auto-CoT*

In the streaming setting on MultiArith, test questions arrive in batches. Auto-CoT* is applied as follows:

- For batch 1, questions are answered with Zero-Shot-CoT and the resulting question–rationale–answer triples are stored.
- For later batches, stored triples form a pool of candidates. Clustering and demonstration sampling are periodically performed over this pool to construct demonstrations for new questions, analogous to Auto-CoT in the full-batch setting.

The accuracy curve across batches shows that:

- For batch 1, Auto-CoT* and Zero-Shot-CoT have identical performance since no demonstrations exist yet.
- From batch 2 onward, Auto-CoT* quickly approaches the performance of Manual-CoT and remains comparable across later batches.

This indicates that Auto-CoT can be extended to more realistic data-stream scenarios without substantial loss of performance.

7 Summary

7.1 Research problem and main contributions

The study addresses the problem of **automatic** chain-of-thought prompting: how to construct effective CoT demonstrations for LLMs using only unlabeled questions and an LLM, without manually written reasoning chains. Manual-CoT requires substantial human effort to design demonstrations, while Zero-Shot-CoT, though powerful, still makes systematic mistakes.

The main contributions are:

- Identification and analysis of **misleading by similarity** in automatic CoT prompting, showing that naive similarity-based retrieval with noisy Zero-Shot-CoT rationales can worsen performance by reinforcing clustered errors.
- Proposal of **Auto-CoT**, a two-stage method that (i) clusters questions to capture semantic diversity, and (ii) constructs one demonstration per cluster by generating Zero-Shot-CoT reasoning chains and applying length-based heuristics.

- Extensive experiments on ten reasoning benchmarks and two LLM backbones (GPT-3 and Codex) demonstrating that Auto-CoT consistently matches or surpasses Manual-CoT while removing the need for manual demonstration design.
- Additional analyses of robustness to wrong demonstrations and an extension **Auto-CoT*** for a streaming setting, showing that cluster-level diversity and simple heuristics mitigate the impact of noisy rationales.

7.2 Innovation vs. prior methods

Compared with Manual-CoT, Auto-CoT replaces human-designed demonstrations with automatically generated question–rationale–answer triples constructed from the LLM itself, organized by clustering and filtered by heuristics. The method operates in a more challenging setting where only test questions are available and no labeled training data are used.

Compared with retrieval-based ICL that assumes reliable labeled examples, Auto-CoT explicitly accounts for the imperfections of Zero-Shot-CoT. It avoids purely similarity-based selection that can propagate errors and instead emphasizes diversity across clusters. This combination of unsupervised clustering, LLM-generated rationales, and heuristic filtering distinguishes Auto-CoT from prior CoT prompting and ICL methods.

7.3 Limitations and future directions

The paper also acknowledges several limitations:

- The clustering quality depends on the Sentence-BERT embeddings and the chosen number of clusters. Alternative representations or adaptive cluster selection might further improve performance.
- Auto-CoT still inherits errors from Zero-Shot-CoT-generated rationales. The method reduces, but does not eliminate, the presence of wrong demonstrations. Integrating explicit verifier models for rationale checking could improve reliability.
- The evaluation is conducted on English-language datasets and a restricted set of LLMs. Generalization to other languages, domains, and model families remains to be established.
- The current method is purely prompt-based and does not modify LLM parameters. Combining Auto-CoT-style prompting with fine-tuning or reinforcement learning could lead to models that internalize patterns discovered through automatic prompting.

7.4 Key takeaways

The following points summarize the essential insights of the paper:

- **Automatic CoT prompting is feasible.** With question clustering, Zero-Shot-CoT rationales, and simple heuristics, Auto-CoT can match or exceed the performance of strong Manual-CoT prompts across multiple reasoning benchmarks.
- **Diversity is crucial under noisy rationales.** Similarity-based retrieval, if used with imperfect Zero-Shot-CoT rationales, can be misleading. Cluster-level diversity in demonstrations is a key design principle for robust automatic CoT prompting.

- **Auto-CoT offers a practical path to scalable reasoning.** By removing the need for manual CoT demonstrations and adapting naturally to streaming settings, Auto-CoT provides a scalable framework for deploying CoT-style reasoning in large language models.